

Recommended OpenWebUI + vLLM Setup for NemoClaw

For persistent NemoClaw and OpenWebUI use, run vLLM with OpenAI-compatible serving enabled, auto tool choice enabled, and the tool-call parser that matches your model family.

OpenWebUI will use vLLM as its model backend. NemoClaw can also connect directly to the same vLLM endpoint.

1. Start vLLM with tool calling enabled

For Hermes 3 style models, use a vLLM command like this:

```
vllm serve /models/Hermes-3-Llama-3.1-8B
--served-model-name hermes-3-llama-3.1-8b
--enable-auto-tool-choice
--tool-call-parser hermes
--host 0.0.0.0
--port 8000
```

This exposes an OpenAI-compatible API at:

```
http://localhost:8000/v1
```

2. Docker Compose example for vLLM + OpenWebUI

```
services:
  vllm-nemocl原因:
    image: vllm/vllm-openai:latest
    container_name: vllm-nemocl原因
    restart: unless-stopped
    ports:
      - "8002:8000"
    volumes:
      - /path/to/models:/models:ro
      - /path/to/hf-cache:/root/.cache/huggingface
    ipc: host
    deploy:
      resources:
        reservations:
          devices:
            - capabilities: [gpu]
              count: all
    environment:
```

```

- VLLM_API_KEY=${VLLM_API_KEY}
command: >
  --model /models/Hermes-3-Llama-3.1-8B
  --served-model-name hermes-3-llama-3.1-8b
  --enable-auto-tool-choice
  --tool-call-parser hermes
  --host 0.0.0.0
  --port 8000
  --gpu-memory-utilization 0.20
  --max-model-len 32768
  --api-key ${VLLM_API_KEY}

openwebui:
  image: ghcr.io/open-webui/open-webui:main
  container_name: openwebui
  restart: unless-stopped
  ports:
    - "3000:8080"
  volumes:
    - openwebui-data:/app/backend/data
  environment:
    - OPENAI_API_BASE_URL=http://vllm-nemoclave:8000/v1
    - OPENAI_API_KEY=${VLLM_API_KEY}
  depends_on:
    - vllm-nemoclave

volumes:
  openwebui-data:

```

After starting the stack, open OpenWebUI at:

```
http://localhost:3000
```

The model should appear as:

```
hermes-3-llama-3.1-8b
```

3. Test the vLLM endpoint from the host

If vLLM is mapped as `8002:8000`, test it with:

```
curl http://localhost:8002/v1/models
-H "Authorization: Bearer ${VLLM_API_KEY}"
```

Expected result: the response should list the served model name:

```
hermes-3-llama-3.1-8b
```

4. Configure OpenWebUI

In OpenWebUI, set the OpenAI-compatible connection to:

```
Base URL: http://vllm-nemocl原因:8000/v1
API Key:  your VLLM_API_KEY
Model:    hermes-3-llama-3.1-8b
```

If configuring from the host instead of inside Docker, use:

```
Base URL: http://localhost:8002/v1
```

If configuring from another container in the same Docker Compose network, use:

```
Base URL: http://vllm-nemocl原因:8000/v1
```

5. Onboard NemoClaw against the same vLLM endpoint

For NemoClaw, connect directly to vLLM rather than going through OpenWebUI:

```
NEMOCLAW_PROVIDER=custom
NEMOCLAW_ENDPOINT_URL=http://localhost:8002/v1
NEMOCLAW_MODEL=hermes-3-llama-3.1-8b
COMPATIBLE_API_KEY=$VLLM_API_KEY
nemocl原因 onboard --non-interactive
```

If the endpoint does not require authentication, set a non-empty placeholder:

```
COMPATIBLE_API_KEY=dummy
```

6. Recommended architecture

```
OpenWebUI
-> vLLM OpenAI-compatible endpoint
-> Hermes 3 model

NemoClaw
-> same vLLM OpenAI-compatible endpoint
-> same Hermes 3 model
```

This keeps OpenWebUI and NemoClaw independent while sharing the same local inference backend.

7. Notes

Use `--tool-call-parser hermes` for Hermes-style models.

Use a different parser if you switch to a different model family.

Keep `--enable-auto-tool-choice` enabled for NemoClaw-style agent/tool workflows.

If OpenWebUI can chat with the model but NemoClaw tool calls fail, check the vLLM parser and model compatibility first.